



KOGNITO

AI LABS

Documentación Completa de KognitoAI

[Un proyecto de Kognito AI Labs](#)

Este documento ofrece una explicación exhaustiva sobre las funciones, estructura, lógica y espíritu del proyecto KognitoAI. KognitoAI es un sistema modular de asistente inteligente diseñado para integrar diversas funcionalidades de inteligencia artificial y gestión de información personal a través de múltiples plataformas, como Telegram y una aplicación web. El objetivo de esta documentación es detallar la arquitectura del proyecto, sus componentes clave, la lógica operativa detrás de su construcción y la visión que impulsa su desarrollo.

Espíritu de KognitoAI

KognitoAI se fundamenta en los principios de modularidad e independencia de plataforma, buscando proporcionar una experiencia unificada y extensible para los usuarios sin importar la interfaz que utilicen. El sistema centraliza la gestión de datos del usuario —que incluye notas, agendas, documentos y memoria— y expone estas funcionalidades a través de una API robusta. Esta API actúa como la columna vertebral para las interacciones mediante un bot de Telegram, una aplicación web y, potencialmente, otras plataformas en el futuro.

La visión detrás de KognitoAI es crear un asistente seguro, escalable y centrado en el usuario que recuerde interacciones previas, gestione datos personales de manera efectiva y aproveche la inteligencia artificial para ofrecer insights proactivos y asistencia personalizada. Un elemento clave de esta visión es el concepto de identidad universal para cada usuario (a través de un `account_id` único), garantizando consistencia de datos entre plataformas. El proyecto enfatiza una arquitectura desacoplada donde la lógica de negocio principal permanece independiente de la interfaz de usuario, permitiendo flexibilidad y facilidad de expansión.

Características Principales

- **Identidad Universal:** Cada usuario recibe un `account_id` único (UUID), independiente de la plataforma, asegurando una gestión de datos coherente en Telegram, web y otros interfaces.
- **Bot de Telegram:** Una interfaz conversacional completa con soporte para comandos, gestión de documentos (subida, consulta, eliminación), notas, agenda y recordatorios.
- **API Centralizada (FastAPI):** Un backend que expone endpoints para chat, autenticación JWT, gestión de documentos, notas, agenda y más, funcionando como el cerebro central del sistema.
- **WebApp / Panel de Usuario:** Una interfaz web para interactuar con el asistente, gestionar documentos y ver notas, autenticada mediante JWT o login con Telegram.
- **Memoria a Largo Plazo y RAG:** Utiliza PostgreSQL con la extensión `pgvector` para almacenar memorias vectoriales y perfiles estructurados, permitiendo al agente de IA recordar conversaciones y documentos relevantes.
- **Herramientas LangChain:** Un conjunto de herramientas personalizadas para el agente de IA, incluyendo operaciones CRUD para notas, gestión de agenda, scraping web, generación de imágenes y manejo de documentos, todas operando con el `account_id` universal.
- **Autenticación Universal:** Soporte para login vía Telegram (para el bot) y autenticación JWT para la webapp y la API.
- **Arquitectura Modular:** Separación clara de responsabilidades en directorios clave como `core/` (lógica de negocio), `telegram_client/` (lógica específica de Telegram), `tools/` (herramientas LangChain), `utils/` (utilidades generales) y la aplicación web frontend.

Estructura del Proyecto

El proyecto KognitoAI está organizado en directorios específicos, cada uno con un rol definido en el sistema general. A continuación, se presenta una

visión detallada de la estructura y propósito de cada componente principal, con un enfoque en cómo está construido el sistema.

Archivos del Directorio Raíz

- **.env.local:** Contiene variables de configuración específicas del entorno para desarrollo local, como claves de API, URLs de bases de datos y otros ajustes sensibles.
- **docker-compose.yml:** Define y ejecuta aplicaciones Docker multi-contenedor, especificando servicios, redes y volúmenes para los componentes del proyecto (backend, bot de Telegram, etc.).
- **Dockerfile.core, Dockerfile.frontend, Dockerfile.telegram, Dockerfile.webapp:** Archivos de configuración Docker para construir imágenes de las diferentes partes de la aplicación (backend principal, frontend, bot de Telegram y webapp respectivamente). Cada Dockerfile detalla cómo se empaqueta cada componente, incluyendo dependencias y configuraciones específicas.
- **next.config.mjs:** Archivo de configuración para Next.js, el framework de React utilizado en el frontend, que permite personalizar ajustes de compilación, runtime y desarrollo.
- **nginx.conf:** Configuración para Nginx, utilizado probablemente como proxy inverso o servidor web en el despliegue del proyecto.
- **package.json, package-lock.json:** Manifiestos del proyecto Node.js que detallan dependencias, scripts y metadatos del proyecto, esenciales para la gestión del frontend.
- **postcss.config.js:** Configuración para PostCSS, una herramienta para transformar CSS con plugins de JavaScript, frecuentemente usada con Tailwind CSS en el frontend.
- **requirements.txt, requirements.webapp.txt:** Listas de dependencias de Python para el proyecto completo y específicamente para el componente webapp, asegurando que el entorno backend esté correctamente configurado.
- **run_api.py, run_telegram_bot.py, run_telegram_panel.py:** Scripts de Python que actúan como puntos de entrada para iniciar el servidor API (FastAPI), el bot de Telegram y el panel de Telegram, respectivamente. Estos scripts son cruciales para arrancar los servicios del sistema.
- **tailwind.config.ts:** Configuración para Tailwind CSS, un framework de CSS basado en utilidades, definiendo estilos personalizados, temas y plugins para el frontend.

- **tsconfig.json**: Archivo de configuración de TypeScript, especificando opciones del compilador y estructura del proyecto para los archivos TypeScript del frontend.

Directorio Core (`core/`)

Este directorio contiene la lógica de negocio principal del sistema, diseñada para ser independiente de cualquier interfaz específica. Aquí se gestionan los datos del usuario y las funcionalidades clave del asistente.

- **agenda_manager.py**: Gestiona funcionalidades relacionadas con la agenda, como la programación y seguimiento de eventos. Este módulo interactúa con la base de datos para almacenar y recuperar información de eventos del usuario.
- **agent.py**: Contiene la lógica central del agente de IA, que es el núcleo de las capacidades de automatización e inteligencia del sistema. Este archivo probablemente define cómo el agente procesa entradas, utiliza herramientas LangChain y genera respuestas basadas en el contexto y la memoria.
- **config.py**: Almacena configuraciones del sistema, como conexiones a bases de datos, claves de API y otros parámetros esenciales para el funcionamiento del backend.
- **database.py**: Maneja las interacciones con la base de datos, incluyendo la configuración de conexiones, ejecución de consultas y gestión de datos. Este módulo es fundamental para la persistencia de datos del usuario.
- **memory_manager.py**: Gestiona la memoria o estado de la aplicación, posiblemente para mantener el contexto en conversaciones o procesos de IA. Utiliza probablemente `pgvector` para almacenar y buscar vectores de memoria, permitiendo al agente recordar interacciones pasadas.
- **notes_manager.py**: Administra funcionalidades de toma de notas, permitiendo la creación, recuperación y modificación de notas asociadas al `account_id` del usuario.
- **reminders_manager.py**: Maneja la configuración y activación de recordatorios, integrándose con otros módulos para notificar a los usuarios en el momento adecuado.

Directorio Público (`public/`)

- **logo-completo.png, logo-simple.png:** Archivos de imagen para los logotipos del proyecto, utilizados en la interfaz de usuario y branding.

Directorio Fuente (`src/`)

Este directorio contiene el código fuente del frontend de la aplicación web, construido con Next.js, un framework de React.

Subdirectorio App (`src/app/`)

- **globals.css:** Estilos CSS globales para la aplicación, definiendo la apariencia general y consistencia visual.
- **layout.tsx:** Define la estructura de diseño general para la aplicación Next.js, sirviendo como base para todas las páginas.
- **(dashboard)/layout.tsx:** Diseño específico para la sección del dashboard de la aplicación, organizando componentes como barras laterales y áreas de contenido.
- **(dashboard)/page.tsx:** Componente de la página principal del dashboard, que probablemente renderiza la vista inicial del usuario autenticado.
- **(dashboard)/chat/[id]/page.tsx:** Página para vistas de chat individuales dentro del dashboard, permitiendo a los usuarios interactuar con el asistente de IA en un contexto específico.
- **(dashboard)/rag/columns.tsx:** Define estructuras de columnas, probablemente para una tabla de datos en la funcionalidad de RAG (Retrieval-Augmented Generation), que permite buscar y gestionar documentos.
- **(dashboard)/rag/data-table.tsx:** Componente para renderizar una tabla de datos en la sección RAG, mostrando información de documentos o resultados de búsqueda.
- **(dashboard)/rag/page.tsx:** Página principal para la funcionalidad RAG dentro del dashboard, integrando componentes para subir, ver y gestionar documentos.
- **(dashboard)/rag/upload-document-dialog.tsx, preview-document-dialog.tsx, edit-document-dialog.tsx, delete-confirm-dialog.tsx:** Componentes de diálogo para interactuar con documentos (subida, vista previa, edición y eliminación), mejorando la experiencia de usuario en la gestión de documentos.
- **login/page.tsx:** Componente de la página de inicio de sesión para autenticación de usuarios, probablemente integrando opciones de login

con Telegram o credenciales.

Subdirectorío Components ([src/components/](#))

- **AppShell.tsx**: Probablemente un componente de carcasa de la aplicación que organiza la estructura general de la interfaz de usuario.
- **Sidebar.tsx**: Componente de interfaz para la barra lateral, utilizado para navegación dentro del dashboard.
- **ThemeProvider.tsx, ThemeToggle.tsx**: Componentes para gestionar y alternar temas visuales (claro/oscuro) en la aplicación.
- **MarkdownRenderer.tsx, InlineMarkdownRenderer.tsx**: Componentes para renderizar contenido en formato Markdown, útiles para mostrar respuestas del asistente o documentación.
- **ui/**: Contiene componentes de interfaz reutilizables de una librería como shadcn/ui, incluyendo `alert-dialog.tsx` , `button.tsx` , `card.tsx` , `dialog.tsx` , `dropdown-menu.tsx` , `form.tsx` , `input.tsx` , `label.tsx` , `resizable.tsx` , `scroll-area.tsx` , `sonner.tsx` , `table.tsx` , `textarea.tsx` , `toast.tsx` , `toaster.tsx` , entre otros, que estandarizan la interfaz de usuario.

Subdirectorío Contexts ([src/contexts/](#))

- **AuthContext.tsx**: Proporciona contexto de autenticación para gestionar el estado de login del usuario y permisos a través de la aplicación, esencial para la seguridad y personalización.

Subdirectorío Hooks ([src/hooks/](#))

- **use-toast.ts**: Hook personalizado para gestionar notificaciones toast en la aplicación, mejorando la retroalimentación al usuario.

Subdirectorío Lib ([src/lib/](#))

- **api.ts**: Contiene funciones o configuraciones para interacciones con la API, facilitando la comunicación entre el frontend y el backend FastAPI.
- **utils.ts**: Funciones de utilidad usadas a lo largo de la aplicación frontend, como formateo de datos o helpers de UI.

Directorío Telegram Client ([telegram_client/](#))

Este directorio implementa la interfaz específica para Telegram, traduciendo interacciones de usuarios a llamadas a la lógica central o herramientas.

- **bot_manager.py**: Gestiona las operaciones e interacciones del bot de Telegram, inicializando el bot y coordinando handlers.
- **notification_scheduler.py**: Programa notificaciones para ser enviadas a través del bot de Telegram, integrándose con módulos como `reminders_manager.py`.
- **tools.py**: Funciones o herramientas de utilidad específicas para el cliente de Telegram.
- **handlers/**: Contiene scripts de manejo para diferentes tipos de interacciones en Telegram:
 - **admin_handlers.py**: Maneja comandos o acciones específicas de administradores.
 - **callback_query_handler.py**: Gestiona consultas de callback desde botones inline o menús.
 - **command_handlers.py**: Procesa entradas de comandos de los usuarios.
 - **document_handlers.py**: Maneja subidas o interacciones con documentos.
 - **message_handlers.py**: Procesa mensajes entrantes de los usuarios, traduciéndolos a acciones del sistema.

Directorio Telegram Panel ([telegram_panel/](#))

- **index.html, script.js, style.css**: Archivos para un panel de control basado en web o interfaz para gestionar el bot de Telegram, proporcionando una alternativa visual a la interacción por comandos.

Directorio Tools ([tools/](#))

Este directorio contiene scripts de Python para funcionalidades específicas o integraciones, utilizados por el agente de IA o el sistema central. Cada herramienta está diseñada para ser agnóstica a la plataforma, operando con el `account_id` universal.

- **add_note_tool.py, delete_note_tool.py, update_note_tool.py, get_notes_tool.py**: Herramientas para la gestión de notas (crear, eliminar, actualizar y obtener).

- **analyze_text_for_insights_tool.py**: Analiza texto para derivar insights, probablemente para características impulsadas por IA.
- **cancel_event_tool.py, schedule_event_tool.py, get_agenda_tool.py**: Herramientas para la gestión de eventos y agenda.
- **delete_document_tool.py, get_document_content_tool.py, get_document_list_tool.py, update_document_metadata_tool.py**: Herramientas para la gestión de documentos.
- **get_proactive_insights_tool.py, proactive_knowledge_linker_tool.py**: Herramientas para generar insights proactivos y vincular conocimiento.
- **github_repo_tool.py**: Integración con GitHub para interacciones con repositorios.
- **image_generation_tool.py**: Herramienta para generar imágenes, posiblemente usando modelos de IA.
- **knowledge_analysis_tool.py**: Analiza conocimiento almacenado para proporcionar respuestas o insights.
- **memory_add_tool.py**: Añade datos a la memoria o contexto de la aplicación para el agente de IA.
- **set_reminder_tool.py**: Configura recordatorios para los usuarios.
- **update_user_profile.py**: Actualiza información del perfil del usuario.
- **web_scraper_tool.py, web_search_tool.py**: Herramientas para scraping de contenido web y realización de búsquedas en la web.

Directorio Utils (`utils/`)

- **analyze_text_for_insights.py**: Utilidad para análisis de texto, similar a la herramienta pero posiblemente más genérica.
- **db_session.py**: Gestiona sesiones o conexiones a la base de datos, facilitando interacciones seguras y eficientes.
- **document_parser.py**: Parsea documentos para extracción o procesamiento de contenido.
- **embeddings.py**: Maneja embeddings, probablemente para tareas de machine learning o procesamiento de lenguaje natural (NLP).
- **helpers.py**: Funciones de ayuda generales usadas a través del proyecto.
- **image_generation.py**: Utilidad para procesos de generación de imágenes.
- **paginator.py**: Proporciona funcionalidad de paginación para listas o conjuntos de datos.
- **security.py**: Contiene funciones relacionadas con seguridad, como encriptación o verificaciones de autenticación.

Lógica y Construcción del Sistema

La lógica de KognitoAI se basa en una arquitectura desacoplada donde la lógica de negocio principal (en `core/`) maneja la gestión de datos y funcionalidades de IA de manera independiente a la interfaz de usuario. Esta separación permite operaciones agnósticas a la plataforma, con la misma lógica backend sirviendo tanto al bot de Telegram como a la aplicación web a través de un backend centralizado FastAPI.

Gestión de Datos del Usuario

Todos los datos del usuario (notas, documentos, agenda, memoria) están vinculados a un `account_id` universal, asegurando consistencia entre plataformas. Los módulos en `core/` como `notes_manager.py`, `agenda_manager.py` y `memory_manager.py` manejan operaciones CRUD para estos datos, interactuando con una base de datos PostgreSQL mejorada con `pgvector` para almacenamiento y recuperación de vectores. Esta capacidad de vectorización permite búsquedas semánticas avanzadas, esenciales para funcionalidades como RAG (Retrieval-Augmented Generation).

Agente de IA

El archivo `agent.py` en el directorio `core/` es el corazón de la funcionalidad de IA, definiendo cómo el agente procesa entradas del usuario, utiliza herramientas LangChain (definidas en `tools/`) y genera respuestas basadas en contexto y memoria a largo plazo. El agente probablemente emplea modelos de lenguaje (como los de Google Gemini, configurados mediante variables de entorno) para entender y responder a las consultas del usuario, integrando datos de memoria para personalizar las interacciones.

Capa de API

El servidor FastAPI, iniciado mediante `run_api.py`, expone endpoints para todas las funcionalidades del sistema, desde interacciones de chat hasta subidas de documentos, asegurados mediante autenticación JWT. Esta API es consumida tanto por el bot de Telegram como por el frontend web, actuando

como un punto central de comunicación. Los endpoints probablemente están estructurados para manejar solicitudes específicas como `/chat` , `/documents` , `/notes` , etc., cada uno mapeado a funciones en los módulos `core/` .

Interacción en el Frontend

El frontend basado en Next.js (`src/app/`) proporciona una interfaz amigable para interactuar con el sistema, con dashboards, interfaces de chat y capacidades RAG para consulta de documentos. Los componentes están estilizados con Tailwind CSS y utilizan elementos de UI reutilizables del directorio `ui/` . La comunicación con el backend se realiza a través de funciones definidas en `api.ts` , que envían solicitudes HTTP a los endpoints de FastAPI.

Integración con Telegram

El directorio `telegram_client/` maneja la lógica específica de la plataforma Telegram, traduciendo interacciones del usuario (mensajes, comandos, subidas de archivos) a llamadas a la lógica central o a la API. Esto asegura una experiencia conversacional fluida, donde los usuarios pueden interactuar con el asistente mediante texto o comandos, y recibir respuestas formateadas adecuadamente para Telegram (por ejemplo, con Markdown o botones inline).

Flujo de Datos y Comunicación

- Entrada del Usuario:** Un usuario interactúa con el sistema ya sea a través del bot de Telegram (enviando un mensaje o comando) o mediante la webapp (haciendo clic en un botón o enviando un formulario).
- Procesamiento de la Entrada:**
 - En Telegram, los handlers en `telegram_client/handlers/` capturan la entrada y la traducen a una acción, como una llamada a una herramienta en `tools/` o una solicitud a la API.
 - En la webapp, el frontend envía una solicitud HTTP a la API FastAPI mediante funciones en `api.ts` .
- Lógica de Negocio:** La API FastAPI, respaldada por módulos en `core/` ,

procesa la solicitud, interactuando con la base de datos si es necesario (a través de `database.py`) y utilizando el agente de IA (`agent.py`) para generar respuestas o realizar acciones.

4. **Memoria y Contexto:** Durante el procesamiento, `memory_manager.py` recupera contexto relevante (como conversaciones pasadas o documentos) para personalizar la respuesta.
5. **Respuesta al Usuario:** La API devuelve la respuesta al cliente (Telegram o webapp), que la formatea y presenta al usuario.

Tecnologías y Dependencias

- **Backend:** Python, FastAPI, PostgreSQL con `pgvector`, LangChain para herramientas de IA.
- **Frontend:** Next.js (React), TypeScript, Tailwind CSS, componentes de `shadcn/ui`.
- **Integración de Plataforma:** Telegram Bot API.
- **Despliegue:** Docker, Docker Compose, Nginx como proxy inverso.
- **Modelos de IA:** Probablemente modelos de Google Gemini u otros configurados mediante `GOOGLE_API_KEY` y variables relacionadas.

Instalación y Despliegue

KognitoAI puede ser desplegado localmente o mediante Docker. Los pasos clave incluyen:

1. Clonar el repositorio y configurar variables de entorno en un archivo `.env`.
2. Instalar dependencias de Python (`requirements.txt`) y Node.js (`package.json`).
3. Inicializar una base de datos PostgreSQL con `pgvector`.
4. Ejecutar el servidor FastAPI (`run_api.py`) y el bot de Telegram (`run_telegram_bot.py`).
5. Opcionalmente, usar `docker-compose up --build` para un despliegue contenerizado.

Variables de entorno críticas incluyen `TELEGRAM_BOT_TOKEN` , `GOOGLE_API_KEY` , `DATABASE_URL` , y `JWT_SECRET_KEY` , entre otras, que configuran las conexiones y seguridad del sistema.

Contribución y Licencia

Las contribuciones son bienvenidas, con guías para crear ramas de características y enviar pull requests. El proyecto está bajo la Licencia MIT, promoviendo colaboración y uso abierto.

Conclusión

KognitoAI representa un enfoque innovador para la asistencia inteligente, combinando modularidad, insights impulsados por IA y accesibilidad multiplataforma. Su arquitectura asegura escalabilidad y adaptabilidad, mientras que su enfoque en la gestión de datos del usuario y la interacción personalizada subraya su compromiso con mejorar la productividad y el compromiso del usuario.

Este documento será actualizado con detalles de implementación más específicos a medida que se revisen archivos clave para obtener insights más profundos sobre la lógica operativa de componentes individuales.

